

Speech and language technology (DA 3452)

...

Lecture 6 – Aug 13
N-Gram Language Models

TF-IDF: Combine the two ingredients

Final document weight = strength **HERE** × rarity **ELSEWHERE**

Term Frequency (TF)

How strong is the term
in this document?

×

Inverse Document Frequency (IDF)

How rare is the term
across the corpus?

=

TF-IDF weight

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

local frequency × corpus rarity

Example: D_1 = “the batsman hit the ball to the boundary”

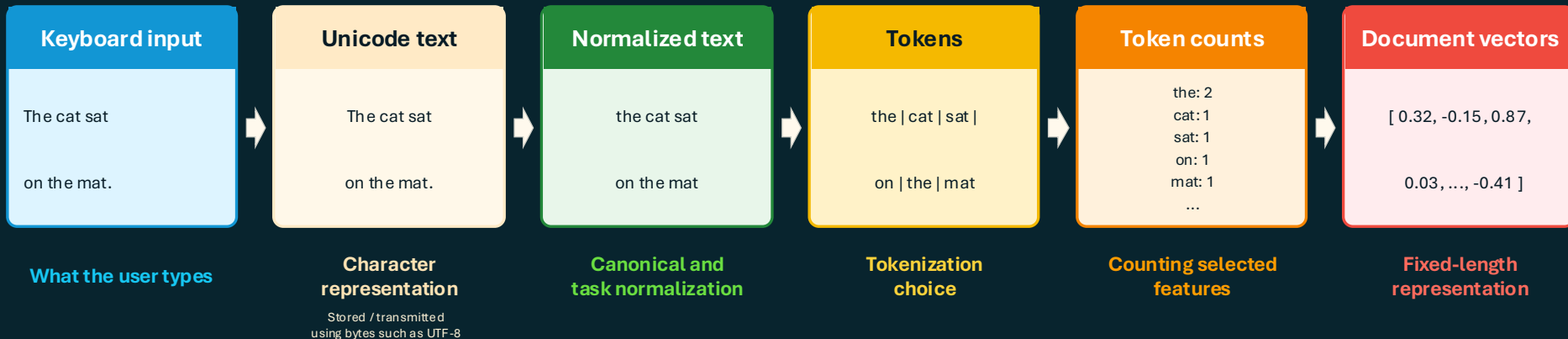
Term	Count $c(t, d)$	TF	DF	IDF	TF-IDF	Interpretation
the	3	1.477	4	0	0	Frequent here, but everywhere
ball	1	1	2	0.301	0.301	Moderately useful
boundary	1	1	1	0.602	0.602	Most discriminative

TF-IDF is high for terms **frequent in this document** but **rare across the corpus**.

Raw counts ask: **how many times?** TF-IDF asks: **frequent here, relative to common elsewhere?**

Module 1 Synthesis: From Input Text to Document Representation

Representation is built step by step — and each step is a modelling choice.



Key modelling choices in Module 1

- 1 Encoding decides how written characters are represented.
- 2 Normalization decides which distinctions are made consistent or removed.
- 3 Tokenization decides what the basic computational units are.
- 4 Weighting decides how much importance each feature receives.

What later models can learn depends on **what this representation makes visible**.

Module 2

Predicting Language and Linguistic Labels

N-Gram Language Models (Ch 3, J&M)

From Representation to Prediction

Module 1 was about Representing language. Module 2 is about Language Prediction.

Document Representation

Document vector



Predict a document label

review → positive / negative

email → spam / not spam

*One whole document is treated
as one object.*

Both are
prediction
problems.

Token Sequence

Token history



Predict the next token

the batsman hit the ____

*The sequence itself becomes
the predictive context.*

We begin Module 2 with the sequence question:

Given previous tokens, what comes next?

What Word Comes Next?

Intuitively, language modelling asks : **Which continuations are more likely.**

the batsman hit the ____

ball

Plausible continuation

boundary

Plausible continuation

moon

Less plausible continuation

democracy

Less plausible continuation

Some continuations feel more natural because language contains regularities.

Some words occur together more often.

Previous words constrain likely next words.

A model can try to learn these regularities from data.

Language modelling: **Predict the next token from the previous tokens.**

An example corpus

A corpus can tell us which continuations have been seen more often.

Small Classroom Corpus

the batsman hit the ball

the batsman hit the boundary

the bowler hit the stumps

the batsman defended the ball

count from
corpus
→

After the context: 'the batsman'

hit	→	2
defended	→	1

Observed more often after this
context → assign greater probability

In Module 1 we counted words in Module 1. We will now count short **word sequences**.

N-gram language modelling begins with a simple idea: **Estimate likelihood from observed counts**.

How Much History Should We Use?

More context can make prediction more specific — but the exact context occurs less often.

Unigram

the batsman hit the

$P(\text{ball})$

No context is used.

Only overall word frequency matters.

Bigram

the batsman hit the

$P(\text{ball} \mid \text{the})$

Use the previous 1 word.

Prediction becomes more
context-sensitive.

Trigram

the batsman hit the

$P(\text{ball} \mid \text{hit the})$

Use the previous 2 words.

Prediction becomes more
specific.

Advantage of More Context

More context can make the prediction
more informative and specific.

Cost of More Context

Longer exact contexts are seen
less often in the corpus.

More history helps prediction — but creates a data sparsity problem.

This tension motivates **n-gram language models** : how much history should we keep?

From 'More Likely' to Probability

Observed continuations can be turned into a probability distribution.

After 'the batsman'

Observed Counts

hit	2
-----	---

defended	1
----------	---

From the small classroom corpus



Normalize
the counts

Convert to Probabilities

$P(\text{hit} \mid \text{the batsman})$	$2/3$
---	-------

$P(\text{defended} \mid \text{the batsman})$	$1/3$
--	-------

$2/3 + 1/3 = 1$

More frequent continuation $\rightarrow$ higher probability.
The probabilities for all possible next tokens must sum to 1.

A language model assigns a **probability distribution** over next tokens given a history.

What Is a Language Model?

We now make next-token prediction mathematically precise.

the batsman hit the —
 $P(\text{ball} \mid \text{the batsman hit the})$

Prediction target

w_t

The token we want to predict

History / context

$w_1, w_2, \dots, w_{t-1}$

The previous tokens

General form

$P(w_t \mid w_1, w_2, \dots, w_{t-1})$

Probability of the next token
given the earlier tokens

A language model estimates probabilities for possible next tokens.

A language model estimates the probability of the **next token** from the **preceding history**.

Probability of a Whole Sentence

A sentence can be viewed as a sequence of next-token prediction decisions.

If a model predicts one word at a time, can it also assign a **probability to an entire sentence**?



Each step asks: **given the history** so far, how likely is the **next word**?

To score a **whole sentence**, we combine the probabilities of successive **next-token choices**.

Building Sentence Probability One Word at a Time

The probability of a whole sentence is built from successive conditional probabilities.

How can we assign a probability to the sequence ‘the batsman hit the ball’?

$P(\text{the batsman hit the ball})$

=

$$P(\text{the}) \times P(\text{batsman} \mid \text{the}) \times P(\text{hit} \mid \text{the batsman}) \times P(\text{the} \mid \text{the batsman hit}) \times P(\text{ball} \mid \text{the batsman hit the})$$

Each factor answers one question:
given the history so far, how likely
is the next word?

Next slide: is this a language-modelling
trick—or a general rule of probability?

The probability of a **sentence** can be built from successive next-token **conditional probabilities**.

The Chain Rule of Probability

the batsman hit the ball

$$\begin{aligned} & P(\text{the batsman hit the ball}) \\ = & P(\text{the}) \\ \times & P(\text{batsman} \mid \text{the}) \\ \times & P(\text{hit} \mid \text{the batsman}) \\ \times & P(\text{the} \mid \text{the batsman hit}) \\ \times & P(\text{ball} \mid \text{the batsman hit the}) \end{aligned}$$

General form

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1})$$

For $i = 1$, the history is empty.

Exact rule

The chain rule is a probability identity,
not yet a modelling approximation.

The chain rule **exactly decomposes** a sequence probability into successive **next-token conditional probabilities**.

Deriving the Chain Rule of Probability

The sentence decomposition is not a language-modelling trick — it follows from repeated use of conditional probability.

Step 1: Start with two events

$$P(B | A) = P(A,B) / P(A) \Rightarrow P(A,B) = P(A) P(B | A)$$

Joint probability = first event × probability of the second given the first.

Step 2: Extend to three events

$$P(A,B,C) = P(A,B) P(C | A,B)$$

$$P(A,B,C) = P(A) P(B | A) P(C | A,B)$$

Apply the same conditional-probability identity again.

Step 3: Replace events by words

$$P(w_1, w_2, w_3) = P(w_1) P(w_2 | w_1) P(w_3 | w_1, w_2)$$

$$P(\text{the batsman hit}) = P(\text{the}) P(\text{batsman} | \text{the}) P(\text{hit} | \text{the batsman})$$

Now the same probability rule is applied to a word sequence.

General chain rule

$$P(w_1, w_2, \dots, w_n) = \prod P(w_i | w_1, \dots, w_{i-1})$$

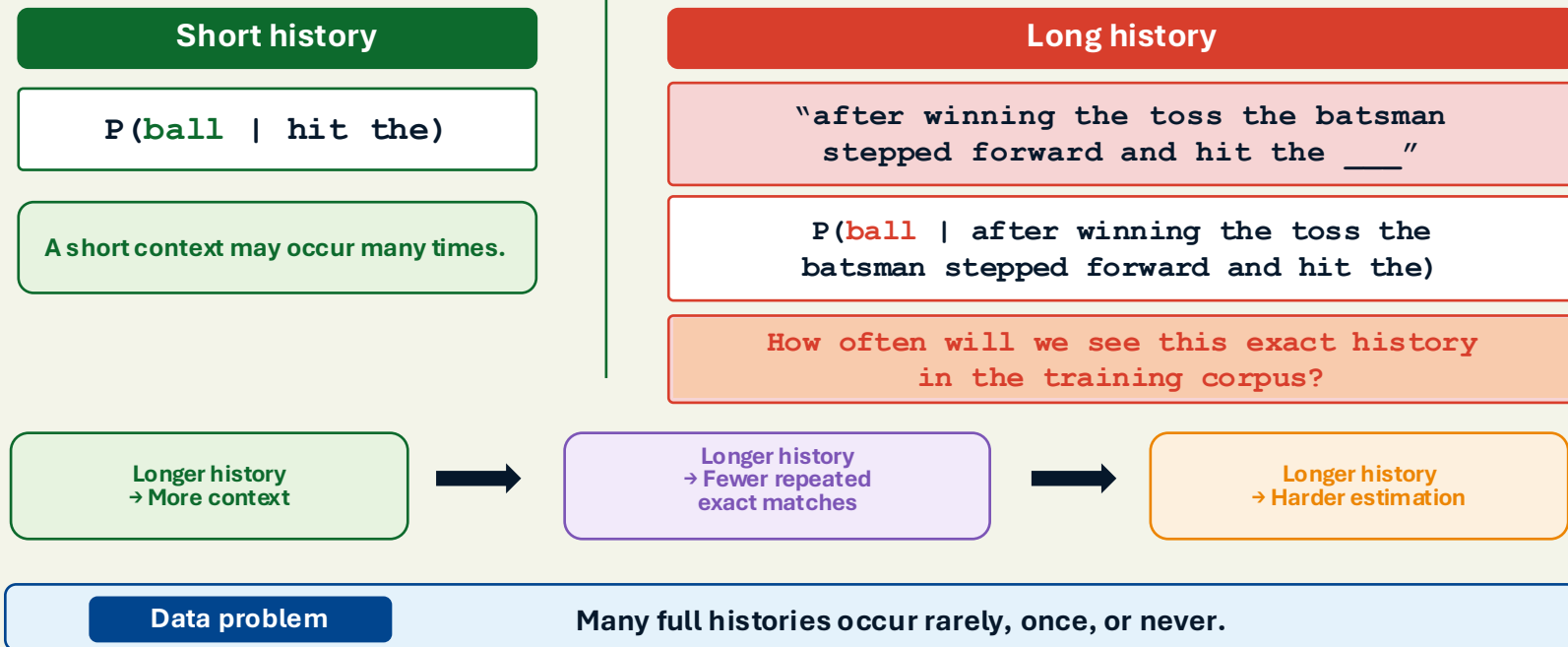
Key point: The chain rule is exact.

The approximation comes later, when we decide how much history to keep.

The chain rule is **exact**; n-gram language models appear only when we **approximate the history**.

Exact vs Easy to Estimate

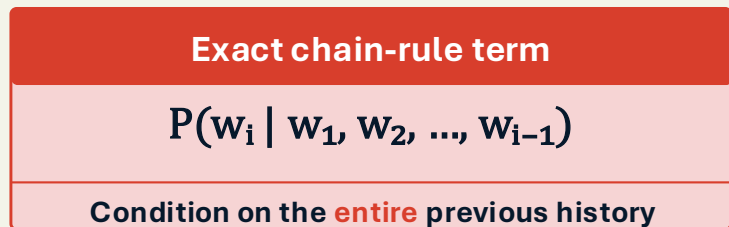
The chain rule is exact — but estimating full-history probabilities from counts quickly becomes impractical.



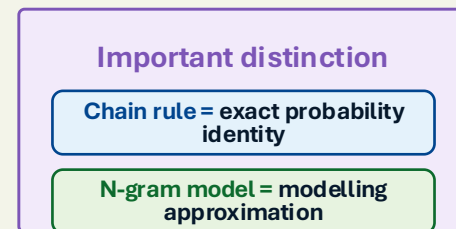
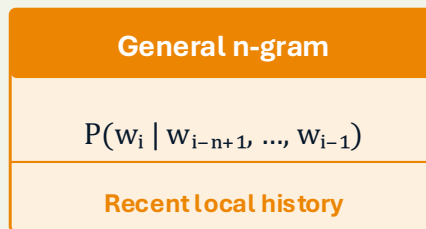
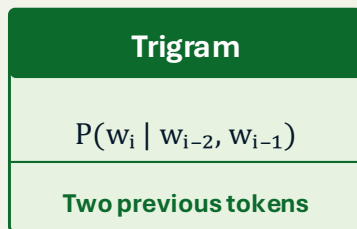
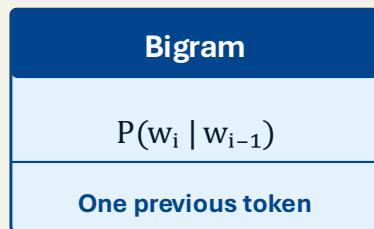
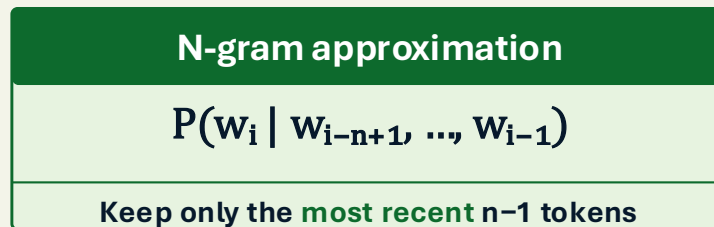
Exact decomposition is not enough: **finite corpora** do not provide reliable counts for most **long histories**.

A Modeling Approximation: Keep Only Recent History

To make estimation practical, we approximate full-history conditioning by keeping **only the most recent tokens**.



Approximate



The **n-gram model** replaces the **full history** with a **short recent history** so that probabilities can be estimated from **finite data**.

Unigram, Bigram, Trigram — Formally

More history gives more context — but also creates many more distinct histories to estimate.

Context example: the batsman hit the ____

Possible next word: ball

Unigram

Formula:

$$P(w_i)$$

Example:

$$P(\text{ball})$$

Interpretation:

Ignore all history

Bigram

Formula:

$$P(w_i | w_{i-1})$$

Example:

$$P(\text{ball} | \text{the})$$

Interpretation:

Use one previous token

Trigram

Formula:

$$P(w_i | w_{i-2}, w_{i-1})$$

Example:

$$P(\text{ball} | \text{hit the})$$

Interpretation:

Use two previous tokens

Trade-off

More history → richer context

More history → sparser counts

As n increases, the model becomes **more context-sensitive** — but the number of **distinct histories** grows and the **data become sparser**.

How Do We Estimate N-gram Probabilities?

For a fixed history, we count possible continuations in the corpus and then normalize those counts into probabilities.

What we want

$$P(w | h)$$

How likely is the next word,
given the previous word?

Maximum-likelihood idea

$$P(w | h) = C(h, w) / C(h)$$

Numerator: count of the n-gram

Denominator: count of the history word

1. Fix a history

Choose the previous
word (the history)
we are conditioning
on.

2. Count the continuations

Count how many times each
word follows that history.

after "the batsman" →
hit: 2
defended: 1

3. Divide by the total for that history

Normalize by the total count
for the history.

$P(\text{hit} | \text{the batsman}) = 2/3$
 $P(\text{defended} | \text{the batsman}) = 1/3$

Why row normalization?

For one fixed history,
the probabilities over all
next words should sum
to 1.

MLE estimates an n-gram probability by turning continuation counts for a fixed history into a probability distribution.

Why Does MLE Normalize the Counts?

For one fixed history, maximum likelihood leads directly to relative-frequency estimates.

Step 1: Fix one history

Suppose the history is h , and the possible next words are

$w_1, w_2, \dots, w_k$.

Let the observed continuation counts be

$c_1, c_2, \dots, c_k$, with $c_j = C(h, w_j)$.

Let the unknown continuation probabilities be

$p_j = P(w_j | h)$, with constraint $\sum_{j=1}^k p_j = 1$.

We are estimating a probability distribution over next words after one fixed history.